kernel 2/4


GENTLEMEN, IT IS WITH
GREAT PLEASURE TO INFORM YOU THAT
I AM NOW A LINUX
KERNEL PROGRAMMER
[81.813747] rfkill: input handler
[82.912780] Bluetooth: RFCOMM TTY
[82.912789] Bluetooth: RFCOMM soc
[82.912795] Bluetooth: RFCOMM ver
[94.053449] rfkill: input handler
[118.101851] Hello world
0|$

Interagir avec Linux

La ligne de commande : paramètres du noyau

- C'est une chaîne pour la configuration à l'exécution :
  - sans recompilation
  - peut être intégrée, via l'option `CONFIG_CMDLINE`
  - peut être utilisée pour passer des arguments au programme `init`

Interagir avec Linux (suite)

- Nombreuses options du noyau, exemples :
  - `root=/dev/sda1` le système de fichiers racine
  - `console=ttyS0` où écrire les messages du noyau
  - `debug`, `loglevel=7` niveau de verbosité du noyau
  - `usbcore.blinkenlights=true` disponible aussi pour les modules
- Plus de documentation : `Documentation/kernel-parameters.txt`

Appels système

- L'appel système est l'interface fondamentale entre une application et le noyau Linux.
- Généralement accessibles via des fonctions enveloppantes de la libc. Le nom de la fonction
  enveloppante est habituellement identique à celui de l'appel système qu'elle invoque.
- Plus de 450 sur x86, certains sont spécifiques à l'architecture, mais la plupart sont communs.
- Un des composants clés de la maturité de Linux.

Le journal du noyau

- Le noyau stocke les messages dans un buffer circulaire :
  - `/proc/kmsg` pour la sortie brute
  - `/dev/kmsg` pour la lecture structurée des messages
- Par défaut, le journal du noyau est affiché sur la console, voir le paramètre noyau `console=`.
- L'outil `dmesg` (message de diagnostic) permet de lire ces messages.
- Avec `systemd`, `journalctl -k` affiche le journal du noyau.

Modules noyau chargeables (LKM)

Chargement

- Nécessite `CONFIG_MODULES=y`
- `insmod` : insère un fichier `.ko` dans le noyau.
- `modprobe` : charge un module (sans `.ko`) et ses dépendances.
- Les deux gèrent les paramètres de module :
  - `$ insmod ./snd-intel8x0m.ko index=-2`

Déchargement

- Nécessite `CONFIG_MODULE_UNLOAD=y`
- `rmmod` : décharge le module.
- `modprobe -r` : supprime aussi les dépendances inutilisées.

Infos

- `lsmod` : affiche l'état des modules dans le noyau Linux.
- `modinfo` : affiche des informations sur un module du noyau Linux.

Pseudo systèmes de fichiers

Il existe de nombreux pseudo systèmes de fichiers sous Linux, en voici quelques-uns :

- `proc` : (`/proc`) informations sur les processus, données brutes, etc.
- `sysfs` : (`/sys`) informations structurées sur divers sous-systèmes du noyau,
  liées aux `kobjects`
- `devtmpfs` : (`/dev`) nœuds de périphériques peuplés par le noyau

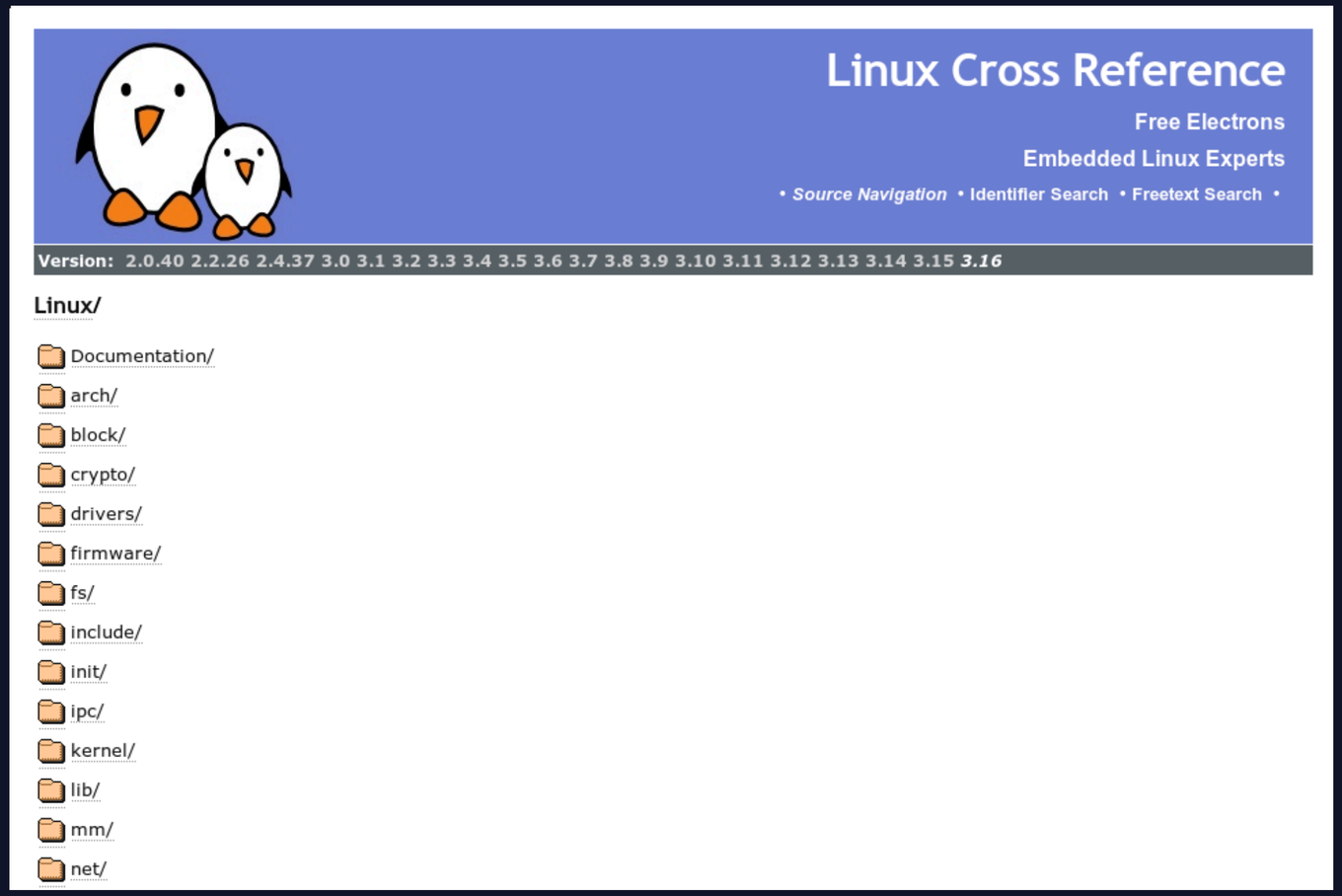


Écrire du code pour Linux

Outils essentiels pour le développeur

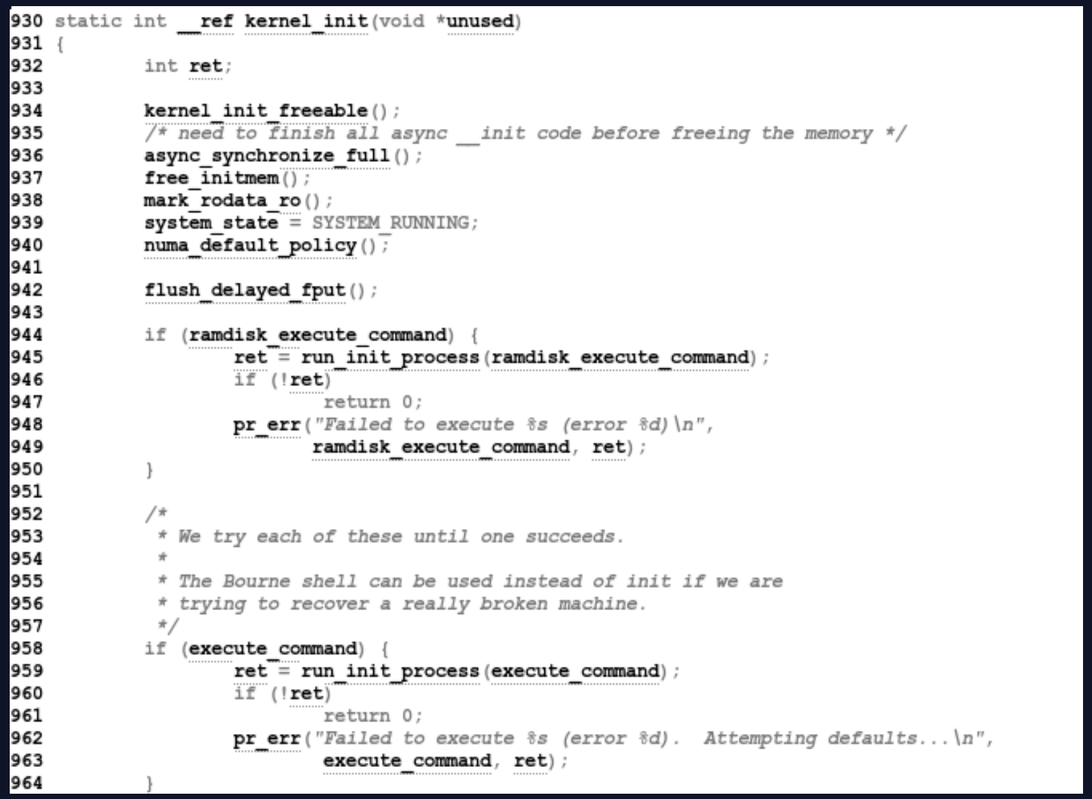
- Le langage C (ISO C99 et extensions GNU)
- GNU Make
- Git

BOOTLIN - Linux X reference



BOOTLIN LXR (https://elixir.bootlin.com/linux/v7.2.6/source)

BOOTLIN - Linux X reference



```
930 static int __ref kernel_init(void *unused)
931 {
932         int ret;
933
934         kernel_init_freeable();
935         /* need to finish all async __init code before freeing the memory */
936         async_synchronize_full();
937         free_initmem();
938         mark_rodata_ro();
939         system_state = SYSTEM_RUNNING;
940         numa_default_policy();
941
942         flush_delayed_fput();
943
944         if (ramdisk_execute_command) {
945                 ret = run_init_process(ramdisk_execute_command);
946                 if (!ret)
947                         return 0;
948                 pr_err("Failed to execute %s (error %d)\n",
949                        ramdisk_execute_command, ret);
950         }
951
952         /*
953          * We try each of these until one succeeds.
954          *
955          * The Bourne shell can be used instead of init if we are
956          * trying to recover a really broken machine.
957          */
958         if (execute_command) {
959                 ret = run_init_process(execute_command);
960                 if (!ret)
961                         return 0;
962                 pr_err("Failed to execute %s (error %d).  Attempting defaults...\n",
963                        execute_command, ret);
964         }
```

BOOTLIN LXR (https://elixir.bootlin.com/linux/v7.2.6/source)

L’espace noyau

- Pas d'accès à la libc habituelle.
- Pas de protection mémoire inviolable.
- Pas d'opérations en virgule flottante.
- Pile de taille fixe (16ko == 4 Pages).
- Préemptif et compatible multiprocesseurs symétriques (SMP). La synchronisation et
  la concurrence sont des préoccupations majeures.

Votre premier code dans le noyau



Où le mettre ?

Deux alternatives :

- Dans l'arborescence officielle du noyau :
  - Intégré dans le dépôt du noyau
  - Peut être compilé statiquement
  - Supporté par la communauté : debug et mise à jour
- Hors arborescence :
  - Dans un répertoire en dehors des sources du noyau
  - Doit être compilé séparément

Hello Kernel (1/3)

Effectuons quelques modifications `in tree` :

- Ajouter une entrée dans linux/init/Kconfig

```
source "init/my_subtree/Kconfig"
```

Hello Kernel (1/3)

Effectuons quelques modifications `in tree` :

- Ajouter une entrée dans linux/init/my_subtree/Kconfig

```
config MY_PATCH
bool "Enable my custom patch"
help
    This is just a test
```

Hello Kernel (2/3)

- Dans init/Makefile :

```
obj-$(CONFIG_MY_PATCH) += my_subtree/myfile.o
```

Hello Kernel (2/3)

- Ajoutons du code :

```
int my_2600_function(void);

int my_2600_function(void)
{
    printk("Hello world\n");
    return 2600;
}
```

Hello Kernel (3/3)

- Patchons `init/main.c`

```
// dans kernel_init
static int __ref kernel_init(void *unused)
{
    int ret;

#if defined(CONFIG_MY_PATCH)
    ret = my_2600_function();
#endif
...
}
```

Hello Module



Créons un fichier `hello.c` .

```c
/* hello.c */
#include <linux/init.h>
#include <linux/module.h>
#include <linux/printk.h>

static __init int hello_init(void)
{
    pr_info("Load the module!\n");
    return 0;
}

static __exit void hello_exit(void)
{
    pr_info("Unload the module.\n");
}

module_init(hello_init);
module_exit(hello_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Hello module");
MODULE_AUTHOR("Lionel Auroux");
```

Explications

Initialisation/nettoyage du module

Macros

- `__init` : pseudo qualifier qui dénote la fonction de chargement du module.
- `__exit` : pseudo qualifier qui dénote la fonction de déchargement du module.
- `module_init(F)` : Appel cette fonction au chargement du module
- `module_exit(F)` : Appel cette fonction au déchargement du module

Explications



Métadonnées du module

- Récupérables avec `modinfo`.
- `MODULE_LICENSE` est important :
  - Les modules non libres n'ont pas accès aux symboles exportés sous GPL.
  - Les modules non libres positionnent le drapeau de contamination propriétaire sur le noyau.
- Aussi utiles : `MODULE_VERSION`, `MODULE_INFO`, `MODULE_SOFTDEP`
- Plus d'informations : `include/linux/module.h`

Explications

- `printk()` est la version noyau de `printf`.
- Plusieurs niveaux de journalisation :
  - `pr_emerg(), pr_alert(), pr_crit(), pr_err(), pr_warn(), pr_notice(), pr_info(), pr_debug()`
- Le code moderne utilise `pr_xxx`.
- `pr_devel()` pendant le développement de votre code.
- `pr_cont()` pour continuer des lignes sans retour à la ligne (`\n`).
- Les formats sont décrits dans `Documentation/printk-formats.txt`.

Explications

`pr_fmt`

Vous définissez la macro `pr_fmt` pour fixer le format par défaut de tous vos appels `pr_xxx`.

```
#define pr_fmt(fmt) KBUILD_MODNAME ": " fmt
#include <linux/printk.h>
...
    pr_devel("Test.\n");
```

Compiler votre module

Vous devez créer un `Makefile`.

Dans votre `Makefile`.

```
# appelé depuis le système de compilation du noyau : déclaration du module
obj-m := hello.o
```

Compiler votre module

- Compilons-le :

```
user $ make -C $Where_Is_My_Linux_Source/ M=`pwd` modules
```

Debug dynamique



- Activé si votre noyau a `CONFIG_DYNAMIC_DEBUG=y`.
- Enregistre un fichier de contrôle : `/sys/kernel/debug/dynamic_debug/control`.
- Affiche la configuration actuelle en lisant ce fichier.
- Activer les appels de debug avec :

```
# debug pour le module `mymodule`
$ cd /sys/kernel/debug/dynamic_debug
$ echo 'module mymodule +p' > control
```

- Documentation dans `Documentation/dynamic-debug-howto.txt`

Plus de documentation

Davantage de documentation

IRC:

- `irc.oftc.net #kernelnewbies`

Web:

- http://kernelnewbies.org

Plus de documentation



Livres:

Attention : Linux évolue vite, ces livres contiennent des informations obsolètes.

- Linux device drivers
- Linux Kernel Development
- Linux System Programming
- Linux in a Nutshell
- Understanding the Linux Kernel
- The Linux Programming Interface

Qu’est-ce qu’un appel système ?

L'espace utilisateur peut émettre des requêtes vers le noyau afin d'accéder à ses ressources ou
d'effectuer des opérations restreintes.

On peut considérer un appel système comme un appel de fonction ordinaire, mais où le code appelé
se trouve dans le noyau.

Exemples d’appel système ?

- Manipulation de fichiers et du VFS : `open`, `read`, `write`, …
- Configuration système : `gettimeofday`, `swapon`, `shutdown`…
- Gestion des processus : `clone`, `mmap`, …
- Manipulation des périphériques : `ioctl`, `mount`, …
- Cryptographie et sécurité : `seccomp`, `getrandom`, …
- …

Les interfaces des appels système côté utilisateur



En assembleur

- Sur x86

```
mov eax, 1 ; exit
int 0x80 ; ou sysenter
```

- Numéro d'appel système : `eax`
- Arguments : `ebx`, `ecx`, `edx`, `esi`, `edi`, `ebp`, puis la pile

Les interfaces des appels système côté utilisateur



En assembleur

- Sur x86_64

```
mov rax, 60 ; exit
syscall
```

- Numéro d'appel système : `rax`
- Arguments : `rdi`, `rsi`, `rdx`, `rcx`, `r8` et `r9`, pas d'arguments en mémoire

syscall(2)

```c
#include <unistd.h>
#include <sys/syscall.h> /* pour __NR_xxx */

long syscall(long number, ...);
```

- Copie les arguments et le numéro d'appel système dans les registres.
- Bascule vers le code noyau.
- Définit `errno` si l'appel système retourne une erreur.

Attention!

- Les appels système peuvent être lents.
- Sur x86 32 bits, `int 0x80` est coûteux : traverse tout le chemin de gestion des interruptions
  dans le microcode du processeur et dans le noyau.
- Même avec une instruction dédiée (`syscall`), une commutation de contexte doit avoir lieu.

Commutation de contexte

Un contexte est :

- Les registres du CPU (incluant le pointeur d'instruction)
- L'état d'un processus (incluant les threads) :
  - État mémoire : pile, tables de pages, etc.
  - État du CPU : registres, caches, etc.
  - État de l'ordonnanceur de processus
  - ...

vdso(7)

- Virtual Dynamically linked Shared Objects
- Petite bibliothèque partagée (8k) que le noyau mappe automatiquement dans l'espace d'adressage de toutes les applications espace utilisateur.
- Contient du code et des données non privilégiés : `gettimeofday`, `time`, `clock_gettime`, ... (dépend de l'architecture)
- L'ELF doit être lié dynamiquement.

Implémentation



- Définir un appel système
  Inclure `#include <linux/syscalls.h>` et utiliser les macros `SYSCALL_DEFINEx(syscall, ...)` n'importe où dans le code Linux.

Ces macros se développent en :

- `SYSCALL_METADATA(syscall, ...)` génère des métadonnées utilisées dans le framework de trace `FTRACE`.
- `__SYSCALL_DEFINEx(syscall, ...)` expansion supplémentaire de la définition de fonction.
- S'étend finalement à : `asmlinkage long SyS_syscall(..)`
- `asmlinkage` signifie que les arguments sont sur la pile.

Exemples

Dans `kernel/signal.c` :

```
3538 SYSCALL_DEFINE0(pause)
3539 {
3540         while (!signal_pending(current)) {
3541                 current->state = TASK_INTERRUPTIBLE;
3542                 schedule();
3543         }
3544         return -ERESTARTNOHAND;
3545 }
```

Exemples

Dans `kernel/signal.c` :

```
#include <asm/current.h>
...
pr_debug("The process is \"%s\" (pid %i)\n",
     current->comm, current->pid);
```

Exemples



Dans `kernel/signal.c` :

```
static inline int signal_pending(struct task_struct *p)
{
    return unlikely(
        test_tsk_thread_flag(p,TIF_SIGPENDING));
}
```

- schedule()

Demande au sous-système d'ordonnancement de choisir le prochain processus à exécuter.

Les tables d’appels système

Voir `arch/x86/entry/syscalls/syscall_{32,64}.tbl`.

- `syscall_32.tbl`

```
# <numéro> <abi> <nom> <point d'entrée> <point d'entrée compat>
0       i386    restart_syscall         sys_restart_syscall
1       i386    exit                    sys_exit
2       i386    fork                    sys_fork                        stub32_fork
3       i386    read                    sys_read
4       i386    write                   sys_write
5       i386    open                    sys_open                        compat_sys_open
6       i386    close                   sys_close
```



`syscall_64.tbl`

```
0       common  read                    sys_read
1       common  write                   sys_write
2       common  open                    sys_open
3       common  close                   sys_close
4       common  stat                    sys_newstat
5       common  fstat                   sys_newfstat
...
16      64      ioctl                   sys_ioctl
...
514     x32     ioctl                   compat_sys_ioctl
```

Génération

- Kbuild appelle `syscalltbl.sh` pour générer `arch/x86/include/generated/asm/syscalls_{64,32}.h`
- De même avec `syscallhdr.sh`
- Si vous ajoutez quelque chose dans `syscall_{32,64}.tbl`, essayez d'ajouter une entrée à la fin.

Tour guidé de quelques appels système



sysinfo

`kernel/sys.c`

```
2099 SYSCALL_DEFINE1(sysinfo,
                     struct sysinfo __user *, info)
2100 {
2101         struct sysinfo val;
2102
2103         do_sysinfo(&val);
2104
2105         if (copy_to_user(info, &val,
                             sizeof(struct sysinfo)))
2106                 return -EFAULT;
2107
2108         return 0;
2109 }
```

`__user`

- Utilisé par des outils comme `sparse` pour vérifier statiquement l'utilisation des pointeurs
  espace utilisateur.
- `# define __user __attribute__((noderef, address_space(1)))`

`copy_to_user`

- Copie des données de l'espace noyau vers l'espace utilisateur.
- Vérifie que tous les octets sont accessibles en écriture, avec :
  - `access_ok(VERIFIY_WRITE, addr_to, length)`

clone

```
SYSCALL_DEFINE5(clone, unsigned long, clone_flags,
                unsigned long, newsp,
                int __user *, parent_tidptr,
                int __user *, child_tidptr,
                int, tls_val)
{
    return do_fork(clone_flags, newsp, 0, parent_tidptr, child_tidptr);
}
```

fork

```
SYSCALL_DEFINE0(fork)
{
    return do_fork(SIGCHLD, 0, 0, NULL, NULL);
}
```

vfork

```
SYSCALL_DEFINE0(vfork)
{
    return do_fork(CLONE_VFORK | CLONE_VM | SIGCHLD, 0,
                   0, NULL, NULL);
}
```

Références

- http://lwn.net/Articles/604287/
- http://lwn.net/Articles/604515/
- https://www.kernel.org/doc/htmldocs/kernel-hacking
- Searchable Linux Syscall Table: https://filippo.io/linux-syscall-table/

TP: Ajouter un syscall — recette pratique



4 fichiers à modifier sur x86_64 :

```
// 1. arch/x86/entry/syscalls/syscall_64.tbl
//    Ajouter à la fin :
451   common  my_syscall   sys_my_syscall

// 2. include/linux/syscalls.h
//    Déclarer le prototype :
asmlinkage long sys_my_syscall(void);

// 3. kernel/my_syscall.c  (nouveau fichier)
#include <linux/syscalls.h>
SYSCALL_DEFINE0(my_syscall)
{
    pr_info("hello from syscall\n");
    return 0;
}

// 4. kernel/Makefile
//    Ajouter :
obj-y += my_syscall.o
```

TP: API temps noyau

Utile pour implémenter `uptime` — ne jamais utiliser `get_seconds()` (déprécié).

```c
#include <linux/ktime.h>

/* Uptime en secondes (simple) */
u64 secs = ktime_get_boottime_seconds();

/* Uptime avec précision nanoseconde */
struct timespec64 ts;
ktime_get_coarse_boottime_ts64(&ts);
// ts.tv_sec  = secondes depuis le boot
// ts.tv_nsec = partie nanoseconde
```

Même source que `/proc/uptime`.

TP: Paramètres de module

Déclarés côté noyau, passés à `insmod` / `modprobe`.

```c
#include <linux/moduleparam.h>

static int my_value = 42;
module_param(my_value, int, 0644);
MODULE_PARM_DESC(my_value, "Une valeur configurable");

// Chargement :
// insmod mon_module.ko my_value=100
// Visible dans /sys/module/mon_module/parameters/my_value
```

Types supportés : `int`, `uint`, `long`, `bool`, `charp` (chaîne), `array`.

TP: `struct file_operations`

Interface entre un périphérique et l'espace utilisateur.

```c
#include <linux/fs.h>

static const struct file_operations my_fops = {
    .owner          = THIS_MODULE,
    .open           = my_open,       /* optionnel */
    .release        = my_release,    /* optionnel */
    .read           = my_read,
    .write          = my_write,
    .unlocked_ioctl = my_ioctl,
};

/* Signatures des callbacks */
static ssize_t my_read (struct file *f, char __user *buf,
                        size_t len, loff_t *off);
static ssize_t my_write(struct file *f, const char __user *buf,
                        size_t len, loff_t *off);
static long    my_ioctl(struct file *f, unsigned int cmd,
                        unsigned long arg);
```

TP: Périphériques misc

Le type de périphérique le plus simple — pas besoin d'allouer un numéro majeur.

```c
#include <linux/miscdevice.h>

static struct miscdevice my_misc = {
    .minor = MISC_DYNAMIC_MINOR,   /* attribué automatiquement */
    .name  = "version",            /* crée /dev/version */
    .fops  = &my_fops,
};

/* Dans la fonction init du module */
static int __init my_init(void)
{
    return misc_register(&my_misc);
}

/* Dans la fonction exit du module */
static void __exit my_exit(void)
{
    misc_deregister(&my_misc);
}
```

TP: Échanges mémoire noyau ↔️ utilisateur 1/2

Toujours utiliser ces helpers — l'accès direct aux pointeurs `__user` provoque un crash.

```
#include <linux/uaccess.h>

/* Noyau → Utilisateur  (dans un handler read ou ioctl) */
if (copy_to_user(user_buf, kernel_buf, size))
    return -EFAULT;
```

TP: Échanges mémoire noyau ↔️ utilisateur 2/2

```
/* Utilisateur → Noyau  (dans un handler write) */
if (copy_from_user(kernel_buf, user_buf, size))
    return -EFAULT;
```

- Retournent le nombre d'octets **non** copiés (0 = succès).
- Vérifient automatiquement que l'adresse utilisateur est valide.

TP: IOCTL — définir des commandes



```c
/* Dans un en-tête partagé (module + programme de test) */
#include <linux/ioctl.h>

#define VERSION_MAGIC   'V'
#define VERSION_MODIFIED  _IOR(VERSION_MAGIC, 0, int) /* lecture */
#define VERSION_RESET     _IO (VERSION_MAGIC, 1)      /* sans données */

/* Implémentation dans le module */
static long my_ioctl(struct file *f,
                     unsigned int cmd, unsigned long arg)
{
    switch (cmd) {
    case VERSION_MODIFIED:
        return copy_to_user((int __user *)arg,
                            &is_modified, sizeof(int));
    case VERSION_RESET:
        /* réinitialiser la version */
        return 0;
    default:
        return -ENOTTY;   /* commande inconnue */
    }
}
```

Macros : `_IO` (sans données), `_IOR` (lecture), `_IOW` (écriture), `_IOWR` (les deux).